Anil Seth

Scientific Python and Image Processing

Discover what a beautiful language Python is for image processing.

A substantial part of the human brain is dedicated to vision and the processing of images. Social sites are full of images that 'friends' share with each other. The sheer number of images from various sites is so huge that aggregation sites like Pixable (*http://pixable.com/*) hope to consolidate them for you, and help you find the ones you want from among all the clutter and noise. Another site, Scalado (*http://scalado.com/*), created an application that allows you to remove unwanted people from your photographs, apart from various other things.

These examples illustrate that algorithms for the better classification or improvement of images and their content is a hot area. Python is a superb tool for exploring ideas and algorithms very quickly. While Python may seem unsuitable for computationally expensive tasks, the Scientific Python (SciPy) community has built tools for fast numerical computation while retaining the power, versatility and flexibility of Python.

A common option for image processing in Python has been the Python Imaging Library (PIL). However, implementing and exploring image-processing algorithms is better done in SciPy or NumPy. There are utility functions that convert a PIL image into a SciPy array and back:

```
im = Image.open('image.png')
A = scipy.array(im)
im2 = Image.fromarray(A)
```

But there is an *ndimage* module in SciPy, which makes it unnecessary to move between the two environments. The advantage of SciPy is that you can manipulate arrays in elegant ways, and not have to manipulate each element in a loop. Getting the sine of each element in an array, A, is as simple as what follows:

```
sinA = scipy.sin(A)
```

As you explore the possibilities with an image, you may find numerous uses for it in other areas as well, including financial modelling *(http://www.logilab.org/blogentry/81704 and http://bit.ly/wLKteD)*!

Fancy indexing

The fancy indexing of NumPy/SciPy arrays provides a powerful abstraction tool. You can think about implementing algorithms at a higher level than manipulating elements of an array. The best approach is for you to actually try these in some simple but realistic examples, and judge for yourself.

You may want to try implementing a threshold on an image, by using a reference image included in SciPy:

```
lena = scipy.lena()
# create an array with zeros of the same size
A = scipy.zeros(lena.shape, dtype='uint8')
# find the elements in lena above the threshold, e.g. 120
mask = lena > 120
# set these to white (255)
A[mask] = 255
```

The use of a mask as an index creates a very flexible and powerful way to manipulate arrays.

You could try a simple method to find a gradient in an image. One such relatively unused method would be to take the difference with the adjacent pixel